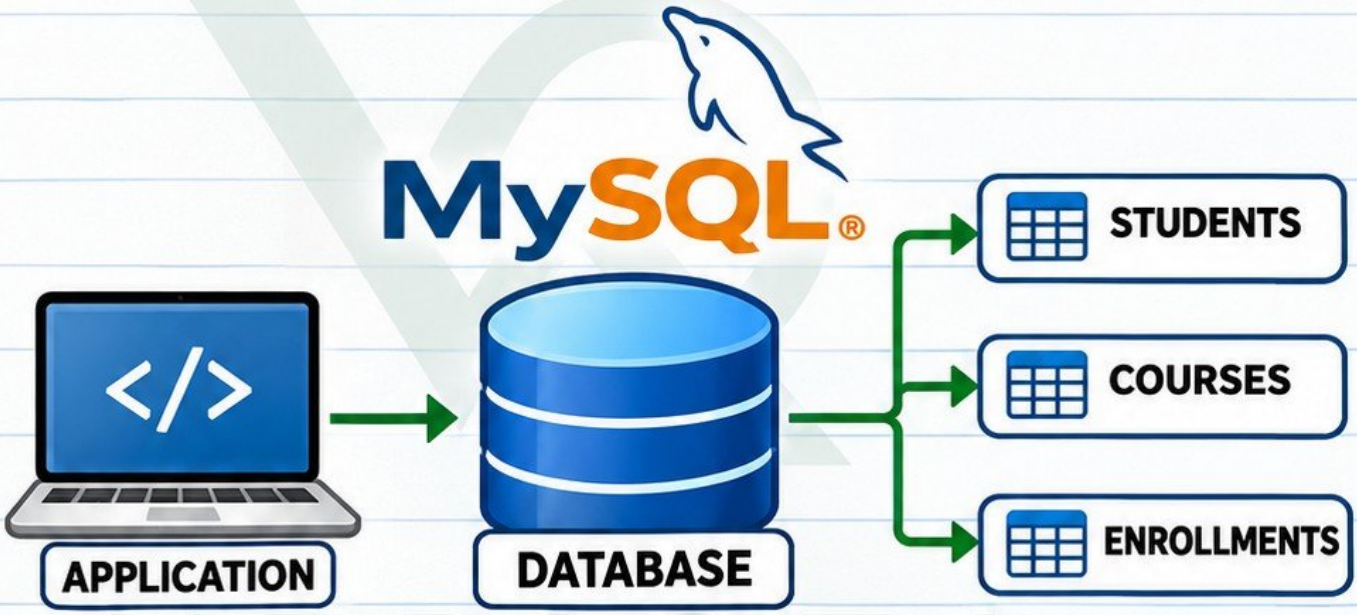
VERIQTA

MySQL for Beginners: Your First Database



Build, Query, Secure, Back Up, and Troubleshoot Your First Real Database

- Understand MySQL and How It Works
- Install, Configure, and Connect to MySQL
- Create Databases and Tables
- Write Practical SQL Queries (CRUD)
- Work With Joins, Indexes, and Transactions
- Secure, Back Up, and Troubleshoot Your Database



@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

What Is MySQL, and Why Do Applications Need It?

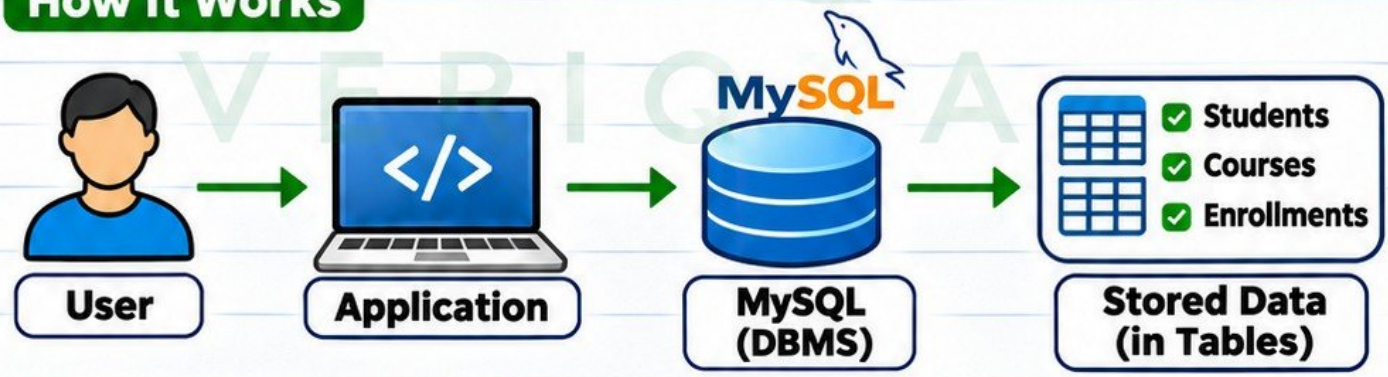
Key Takeaways

- A database stores application data in an organized and persistent way.
- MySQL is a database management system (DBMS).
- SQL is the language used to interact with the database.

Key Terms

- **Database:** A collection of related data (e.g., students, courses).
- **MySQL:** A popular, open-source DBMS that stores and manages data.
- **SQL:** Structured Query Language, used to create, read, update, and delete data.
- **Database Server:** The running service that hosts databases and handles requests from applications.

How It Works



Real-World Example



An online academy (e.g., VERIQTA Academy) uses MySQL to store student registrattions, course details, and enrollments. The application reads, adds, and updates this data using SQL.

Junior Engineer Insight

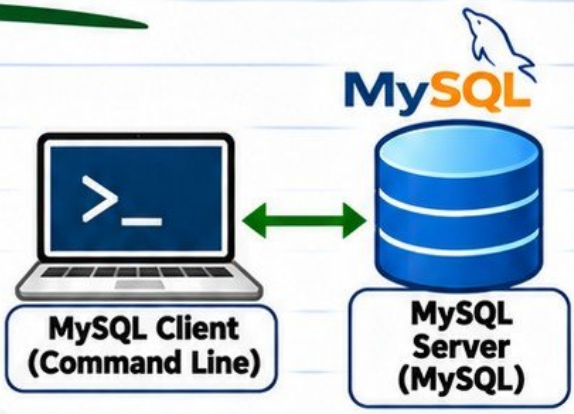


MySQL is a database management system.
SQL is the language used to interact with it.

Install MySQL and Connect to Your First Server

Key Concepts

- **MySQL Server:** The service that stores and manages your databases.
- **MySQL Client:** A command-line tool used to connect to the server and run SQL commands.



Install and Connect (Ubuntu)

1. **Install MySQL**

```
sudo apt update
sudo apt install mysql-server -y
```

2. **Check the service status**

```
sudo systemctl status mysql
```

3. **Connect to MySQL**

```
mysql -u root -p
```

4. **Confirm the connection**

```
SELECT VERSION();
```

Example: Service Status (Running)

```
● mysql.service - MySQL Server
   Active: active (running)
```

Example: Connected to MySQL

```
mysql> SELECT VERSION();
+--------------+
| VERSION()    |
+--------------+
| 8.0.36       |
+--------------+
1 row in set (0.00 sec)
```

Troubleshooting Common Issues

Problem	What It Means	How to Fix
Access denied	Wrong username or password.	■ Verify the username and password. ■ Ensure the user has permission.
Connection refused	The MySQL server is not reachable.	■ Check if the service is running: `sudo systemctl status mysql`

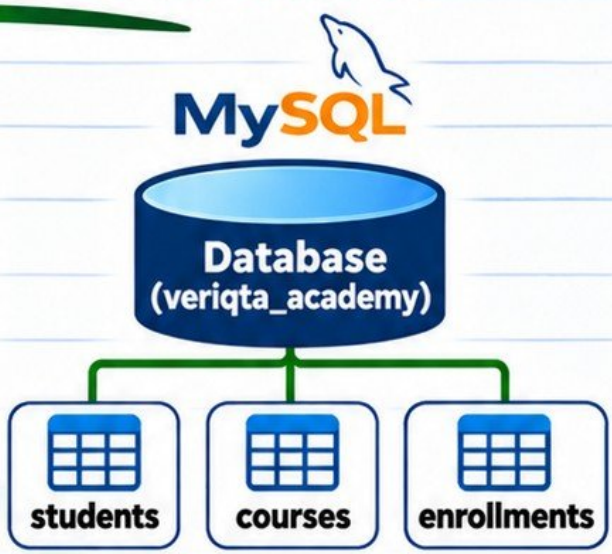
Service not running	MySQL is installed but stopped.	■ Start and enable it:
		`sudo systemctl start mysql`
		`sudo systemctl enable mysql`

Security Reminder | Use a dedicated database account for applications, not the root user.

Create Your First Database

Key Concepts

- **Database:** A container that stores related data (e.g., students, courses).
- **Schema:** The structure or organization of data in a database (used interchangeably with database).
- **Table:** A structured collection of data in rows and columns (e.g., a students table).



Steps: Create and Use a Database

1. Create a database

```
CREATE DATABASE veriqta_academy;
```

2. List available databases

```
SHOW DATABASES;
```

3. Select (use) your database

```
USE veriqta_academy;
```

4. Check tables in the database

```
SHOW TABLES;
```

Example: SHOW DATABASES;

```
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| veriqta_academy    |
+--------------------+
```

Example: SHOW TABLES;

```
+-------------------------+
| Tables_in_veriqta_academy |
+-------------------------+
| (empty set)             |
+-------------------------+
```

Practice

- [x] Create the veriqta_academy database.
- [x] Select it and check for tables.
- [x] Verify that it exists using SHOW DATABASES;.

Beginner Mistake

Creating a database does not automatically create tables.

Design Your First Table: Students

Key Concepts

- A **table** stores data in rows and columns.
- A **row** is one record (e.g., one student).
- A **column** is a field (e.g., email, date).

Common Data Types

Data Type	Use Case (Example)
INT	Whole numbers (e.g., student_id).
VARCHAR	Text with a set length (e.g., name, email).
DATE	A calendar date (e.g., 2026-11-05).
TIMESTAMP	Date and time (e.g., record created time).

Create the Students Table

```
CREATE TABLE students (
  student_id INT PRIMARY KEY
  AUTO_INCREMENT,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  created_at TIMESTAMP DEFAULT
  CURRENT_TIMESTAMP
);
```

Key Constraints Explained

- ✓ **Primary Key:** A unique identifier for each record (e.g., student_id).
- ✓ **AUTO_INCREMENT:** Automatically generates the next number.
- ✓ **NOT NULL:** The field must have a value (e.g., full_name, email).
- ✓ **UNIQUE:** Ensures no duplicate value (e.g., email must be unique).

Example: Students Table (Data)

student_id	full_name	email	created_at
1	Sarah Okafor	sarah@veriqta.com	2026-11-05 10:21
2	James Adeyemi	james@veriqta.com	2026-11-05 10:35
3	Ruth Danjuma	ruth@veriqta.com	2026-11-05 10:40



Practice

- ✓ Create the students table.
- ✓ Run **DESCRIBE students;**
- ✓ Verify the columns and constraints.

Inspect the Table Structure

DESCRIBE students;

```
+------------+--------------+------+-----+---------------+
| Field      | Type         | Null | Key | Extra         |
+------------+--------------+------+-----+---------------+
| student_id | int          | NO   | PRI | auto_increment|
| full_name  | varchar(100) | NO   |     |               |
| email      | varchar(150) | NO   | UNI |               |
| created_at | timestamp    | YES  |     | DEFAULT ...   |
+------------+--------------+------+-----+---------------+
```

Production Awareness

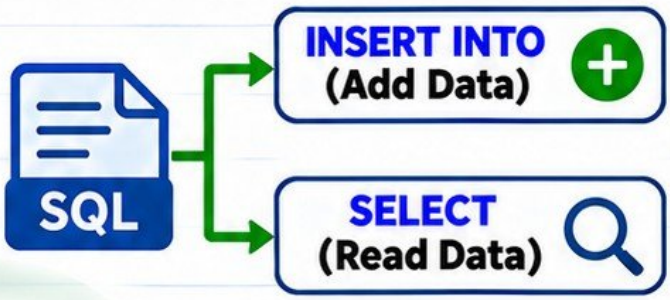


Good table design prevents duplicate and inconsistent data in production.

Insert and Read Your First Student Records

Key Concepts

- **INSERT INTO:** Adds new records (rows) to a table.
- **SELECT:** Retrieves data from a table.
- You can read all or specific columns, and control the order and number of results.



1. Insert Three Student Records

```
INSERT INTO students
  (full_name, email) VALUES
('Sarah Okafor', 'sarah@veriqta.com'),
('James Adeyemi', 'james@veriqta.com'),
('Ruth Danjuma', 'ruth@veriqta.com');
```

2. Read All Records

```
SELECT * FROM students;
```

student_id	full_name	email
1	Sarah Okafor	sarah@veriqta.com
2	James Adeyemi	james@veriqta.com
3	Ruth Danjuma	ruth@veriqta.com

3. Read Specific Columns

```
SELECT full_name, email
FROM students;
```

full_name	email
Sarah Okafor	sarah@veriqta.com
James Adeyemi	james@veriqta.com
Ruth Danjuma	ruth@veriqta.com

4. Control Results (ORDER BY and LIMIT)

```
-- Order by name (A to Z)
SELECT * FROM students
ORDER BY full_name ASC;

-- Return only the first 2 records
SELECT * FROM students
ORDER BY created_at DESC LIMIT 2;
```

Practice

- [x] Insert a new student into the students table.
- [x] Run `SELECT * FROM students;` to verify it.
- [x] Try selecting only full_name and email.

Beginner Mistake

A successful insert does not mean your application is reading the correct database.

Find Exactly the Records You Need

Key Concepts

- **WHERE**: Filters records to return only the ones you need.
- **AND / OR**: Combine multiple conditions.
- **LIKE**: Searches for text patterns (e.g., email domain).
- **IS NULL**: Finds records with missing (NULL) values.

Comparison Operators

Operator	Meaning (Example)
=	Equal to (e.g., student_id = 1)
<>	Not equal to (e.g., status <> 'inactive')
>	Greater than (e.g., score > 70)
<	Less than (e.g., age < 30)
AND	Both conditions must be true
OR	At least one condition must be true
LIKE	Pattern match (e.g., email LIKE '%@veriqta.com')
IS NULL	Value is missing (e.g., phone IS NULL)

SQL Examples With the Students Table

Find one student by ID	`SELECT * FROM students WHERE student_id = 1;`
Find students not from a domain	`SELECT * FROM students WHERE email <> 'school.com';`
Find students with two conditions (AND)	`SELECT * FROM students WHERE full_name LIKE 'Sarah%' AND created_at > '2026-01-01';`
Find students matching any condition (OR)	`SELECT * FROM students WHERE email LIKE '%@veriqta.com' OR email LIKE '%@gmail.com';`
Search by text pattern (LIKE)	`SELECT * FROM students WHERE full_name LIKE '%Ade%';  -- name contains "Ade"`
Find students with missing phone number	`SELECT * FROM students WHERE phone IS NULL;`

Practice



- ✅ Find all students whose email ends with @veriqta.com.
- ✅ Find students created after 2026-06-01.
- ✅ Find students with no phone number.

Example Result

student_id	full_name	email
1	Sarah Okafor	sarah@veriqta.com
3	Ruth Danjuma	ruth@veriqta.com



Troubleshooting Tip | SQL **NULL** is not the same as an empty string (**''**).

Update and Delete Records Without Losing Data

Key Concepts

- **UPDATE:** Changes existing data in a table.
- **DELETE:** Removes specific records from a table.
- **WHERE:** Restricts the change to the intended record(s).
- Always preview with **SELECT** before making changes.

Update & Delete at a Glance



1. Update a Record

```
UPDATE students
SET email = 'sarah.new@veriqta.com'
WHERE student_id = 1;
```

student_id	full_name	email
1	Sarah Okafor	sarah.new@veriqta.com

2. Delete a Record

```
DELETE FROM students
WHERE student_id = 3;
```

student_id	full_name	email
1	Sarah Okafor	sarah@veriqta.com
2	James Adeyemi	james@veriqta.com

DELETE, TRUNCATE, and DROP: What's the Difference?

Command	What It Does	Keeps Table?	Use Case (Example)
DELETE	Removes selected rows (you can use WHERE).	✓ Yes	Delete a specific student (e.g., WHERE student_id = 3).
TRUNCATE	Removes all rows quickly (without a WHERE clause).	✓ Yes	Clear all records in a table (e.g., reset test data).
DROP	Completely removes the table (structure and all its data).	✗ No	Remove the table when it is no longer needed.

Practice



- ✓ Preview the record with **SELECT**.
- ✓ Update a student's email.
- ✓ Verify the change with **SELECT**.
- ✓ Delete a student by ID.
- ✓ Confirm the record is removed.

Critical Warning



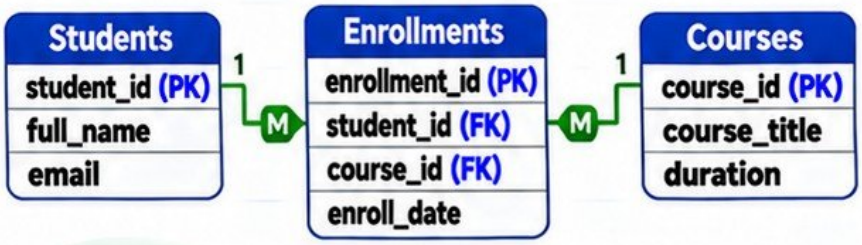
An **UPDATE** or **DELETE** without **WHERE** can affect every row in the table.

Connect Tables With Primary and Foreign Keys

Key Concepts

- Applications use multiple related tables to organize data and avoid duplication.
- A one-to-many relationship means one record in a table can be linked to many records in another table (e.g., one student can have many enrollments).

Data Model: Students, Enrollments, Courses



Students → Enrollments ← Courses

Create the Courses Table

```
CREATE TABLE courses (
  course_id INT PRIMARY KEY
  AUTO_INCREMENT,
  course_title VARCHAR(100) NOT NULL,
  duration VARCHAR(50) NOT NULL
);
```

Create the Enrollments Table

```
CREATE TABLE enrollments (
  enrollment_id INT PRIMARY KEY
  AUTO_INCREMENT,
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  enroll_date DATE NOT NULL,
  FOREIGN KEY (student_id) REFERENCES
  students(student_id),
  FOREIGN KEY (course_id) REFERENCES
  courses(course_id)
);
```

Example: Table Data

students

student_id	full_name
1	Sarah Okafor
2	James Adeyemi

courses

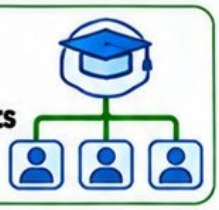
course_id	course_title
1	DevOps Basics
2	Cloud Security

What Are Primary and Foreign Keys?

- **Primary Key (PK):** A unique identifier for each record in a table (e.g., student_id, course_id).
- **Foreign Key (FK):** A column that references the primary key in another table (e.g., enrollments.student_id → students.student_id).
- **Referential Integrity:** Ensures the relationship is valid (you cannot enroll a student or course that does not exist).

One-to-Many Example

- One student can have many enrollments.
- One course can have many students (enrollments).



Beginner Mistake

Storing all student and course information in one giant table creates duplication and inconsistent data.

Retrieve Related Data With SQL JOINs

Key Concepts

- Combine data from multiple related tables using **JOIN**.
- Use **INNER JOIN** to get only matching records.
- Use **LEFT JOIN** to get all records from the left table, even if there is no match in the right table.

Join Types at a Glance

Join Type	What It Does
INNER JOIN	Returns only rows that have matches in both tables.
LEFT JOIN	Returns all rows from the left table (e.g., students) and matching rows from the right table (or NULL if none).

How the JOIN Works

students (s)

student_id	full_name
1	Sarah Okafor
2	James Adeyemi
3	Ruth Danjuma

+

enrollments (e)

student_id	course_id
1	101
2	102

→

JOIN Result (Students and Courses)

student_id	full_name	course_id
1	Sarah Okafor	101
2	James Adeyemi	102

SQL Examples

1. INNER JOIN – Students and Their Courses

```
SELECT s.full_name, c.course_title
FROM students s
INNER JOIN enrollments e
ON s.student_id = e.student_id
INNER JOIN courses c
ON e.course_id = c.course_id;
```

2. LEFT JOIN – List Students Who Have Not Enrolled

```
SELECT s.student_id, s.full_name, c.course_title
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
LEFT JOIN courses c ON e.course_id = c.course_id
WHERE e.course_id IS NULL;
```

Table Aliases & Matching Keys

- A **table alias** gives a short name for a table (e.g., students AS s).
- The JOIN matches related records using the **key columns** (e.g., s.student_id = e.student_id).
- Aliases make queries shorter and easier to read.

Practice



- List all students and their enrolled courses.
- List students who have not enrolled in a course.
- Use table aliases in your queries.

Troubleshooting Tip

Incorrect **JOIN** conditions can produce missing results or duplicate rows.

Calculate Useful Information With SQL

Key Concepts

- Use aggregate functions to calculate information from your data.
- **GROUP BY** groups rows (e.g., by course) so you can calculate per group.
- **HAVING** filters group results (e.g., only courses with 10+ enrollments).
- Count records vs. count unique students are not the same.

Common Aggregate Functions

Function	What It Does (Example)
COUNT()	Counts records (e.g., COUNT(*)).
SUM()	Adds values (e.g., SUM(amount)).
AVG()	Calculates the average value (e.g., AVG(amount)).
MIN()	Returns the smallest value (e.g., MIN(amount)).
MAX()	Returns the largest value (e.g., MAX(amount)).

Sample Tables

enrollments

enroll_id	student_id	course_id
1	1	101
2	2	101
3	3	102
4	1	103

payments

payment_id	student_id	amount
1	1	50000
2	2	50000
3	3	75000

GROUP BY and HAVING

- **GROUP BY:** Groups records (e.g., by course_id) so you can summarize per course.
- **HAVING:** Filters the grouped results (e.g., only courses with 10 or more enrollments).

SQL Examples

1. Count Enrollments Per Course

```
SELECT c.course_title, COUNT(*) AS enrollment_count
FROM enrollments e
JOIN courses c ON e.course_id = c.course_id
GROUP BY c.course_title
ORDER BY enrollment_count DESC;
```

→

course_title	enrollment_count
DevOps Basics	2
Cloud Security	1
Kubernetes	1

2. Calculate Total Payments

```
SELECT SUM(amount) AS total_payments
FROM payments;
```

→

total_payments
175000

3. Identify the Course With the Most Enrollments

```
SELECT c.course_title, COUNT(*) AS enrollment_count
FROM enrollments e
JOIN courses c ON e.course_id = c.course_id
GROUP BY c.course_title
ORDER BY enrollment_count DESC
LIMIT 1;
```

→

course_title	enrollment_count
DevOps Basics	2

Practice



- Count enrollments per course.
- Calculate the total payments received.
- Identify the course with the most enrollments.
- Try counting unique students per course using COUNT(DISTINCT student_id).

Production Connection



Reporting dashboards depend on correctly aggregated database records (e.g., totals, trends, and course statistics).

Choose Data Types That Protect Your Data

Key Concepts

- Choose the right data type to store the right kind of data correctly.
- Use constraints (NOT NULL, DEFAULT, UNIQUE) to keep data valid and consistent.
- Use DECIMAL for money, not FLOAT, to avoid rounding errors.

Common SQL Data Types

Data Type	Use Case	Example
INT	Whole numbers (e.g., IDs, counts).	1, 100, 5000
BIGINT	Very large whole numbers.	> 2 billion
VARCHAR(n)	Short text (up to n characters).	VARCHAR(100)
TEXT	Long text (e.g., notes, descriptions).	"Course details..."
DATE	A date only (YYYY-MM-DD).	2026-11-10
DATETIME	Date and time.	2026-11-10 14:30:00
DECIMAL(p,s)	Exact numeric values (e.g., money).	DECIMAL(10,2)

Common Constraints

Constraint	What It Does
NOT NULL	Value is required (cannot be empty).
DEFAULT	Sets a default value (if no value is provided).
UNIQUE	Ensures all values in the column are different (e.g., email, reference).

Why Use DECIMAL for Money?

DECIMAL (Recommended)

- Stores exact values.
- No rounding errors.
- Reliable for payments, balances, and totals.

FLOAT (Not Recommended)

- Approximate values (can be slightly wrong).
- May cause rounding errors (e.g., 100.00 can be stored as 99.9999).

Design the Payments Table

```
CREATE TABLE payments (
  payment_id INT PRIMARY KEY AUTO_INCREMENT,
  student_id INT NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending',
  payment_date DATETIME NOT NULL DEFAULT NOW(),
  UNIQUE (student_id, payment_date, amount)
);
```

Payments Table (Example Data)

payment_id	student_id	amount	status	payment_date
1	1	50000.00	paid	2026-11-01 10:15:00
2	2	75000.00	pending	2026-11-08 09:30:00
3	1	25000.00	paid	2026-11-10 14:05:00

Practice



- Choose the best data type for:
  - Payment amount → DECIMAL(10,2).
  - Payment status → VARCHAR(20).
  - Payment date → DATETIME.
- Add NOT NULL and a DEFAULT value for the status.
- Ensure a unique rule where needed (e.g., transaction reference).

Junior Engineer Insight



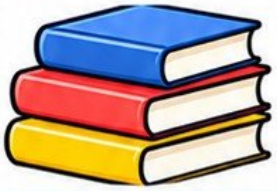
Data types are part of data integrity, not merely storage decisions.

Understand Indexes and Faster Database Searches

Key Concepts

- A database can be slow when it has to **SCAN** every row in a table.
- An **INDEX** is a special data structure that helps the database find data faster.
- Indexes improve **READ** performance but use storage and add **WRITE** overhead (inserts, updates, deletes).

The Lookup Analogy



VS



No Index
(Like reading every page of a book to find a topic)

With Index
(Like using the book's index to go directly to the page)

Create an Index

```
CREATE INDEX idx_students_email
ON students(email);
```

Inspect a Query With EXPLAIN

```
EXPLAIN SELECT *
FROM students
WHERE email = 'sarah@veriqta.com';
```

Full Table Scan vs Indexed Lookup

Full Table Scan (Slower)

students (no index)	
Row 1	✗
Row 2	✗
Row 3	✗
...	
Row N	✓

Indexed Lookup (Faster)



EXPLAIN Output Example

id	select_type	table	type	key	rows
1	SIMPLE	students	ref	idx_students_email	1

- ✓ **type:** How the table is read (ref = using an index).
- ✓ **key:** The index used (idx_students_email).
- ✓ **rows:** Estimated number of rows to check (lower is better).

Pros and Cons of Indexes

Benefits	Trade-offs
✓ Faster searches (SELECT).	✗ Uses additional storage (disk space).
✓ Better performance for frequent lookups.	✗ Adds write overhead (INSERT, UPDATE, DELETE).
✓ Helps with JOINs and filtering (WHERE).	✗ Too many indexes can slow down writes.

Practice



- ✓ Create an index on students.email.
- ✓ Run **EXPLAIN** on a query that uses the index.
- ✓ Observe the **key** and **rows** values.
- ✓ Compare with a query without an index (optional).

Production Awareness

Indexes should support **real query patterns** (e.g., frequent searches and JOINs), not be added to every column.

Transactions: Keep Related Changes Together

Key Concepts

- A **TRANSACTION** groups related database operations so they all succeed or all fail.
- Use **START TRANSACTION** to begin, **COMMIT** to save changes, and **ROLLBACK** to undo changes.
- Useful for actions like enrolling a student and recording their payment together.

Transaction Commands

Command	What It Does
START TRANSACTION	Begins a new transaction.
COMMIT	Permanently saves all changes in the transaction.
ROLLBACK	Undoes all changes made in the transaction.

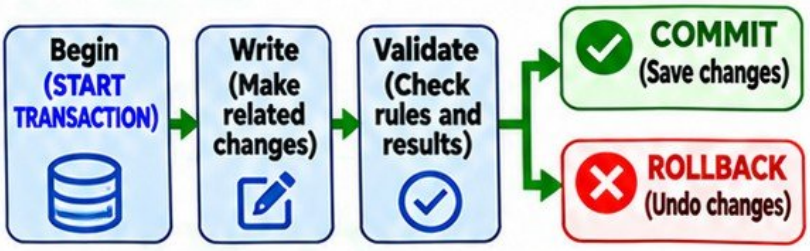
ACID Properties (Transaction Guarantees)

A	Atomicity	All operations succeed or all fail.
C	Consistency	Keeps data valid and follows rules (e.g., foreign keys, constraints).
I	Isolation	Transactions do not interfere with each other.
D	Durability	Committed changes are permanent, even after a system failure.

Example: Enrollment and Payment

```
START TRANSACTION;
INSERT INTO enrollments
(student_id, course_id, enroll_date)
VALUES (1, 101, NOW());
INSERT INTO payments
(student_id, amount, status, payment_date)
VALUES (1, 50000.00, 'paid', NOW());
COMMIT;
```

Transaction Flow



Real-World Example

Operation	Result (With Transaction)
✓ Create enrollment	Saved together.
✓ Record payment	Saved together.
✓ Both succeed	COMMIT (both records kept).
✗ One fails (e.g., invalid course)	ROLLBACK (no record is saved).

Practice



- ✓ Start a transaction.
- ✓ Insert a test enrollment and payment.
- ✓ Roll back the change and verify that no record was saved (SELECT to confirm).

Production Connection

Transactions help prevent **partially completed operations**, keeping your data consistent and reliable.

Understand Database Relationships and Normalization

Key Concepts

- Find and remove **REPEATED** data to avoid errors.
- Use **NORMALIZATION** (1NF, 2NF, 3NF) to organize data properly.
- Separate data into related tables (students, courses, enrollments, payments).
- Use **FOREIGN KEYS** and **UNIQUE** constraints to keep data consistent.

The Normal Forms (Beginner View)

Normal Form	What It Means (Simply)
1NF	Use a table with clear, single values in each column (no repeating lists).
2NF	Remove partial dependencies (non-key columns must depend on the whole key).
3NF	Remove transitive dependencies (e.g., don't store course name in enrollments; keep it in the courses table).

Before and After: Table Design

Before: One Big Table (Problematic)

student_id	student_name	course	amount
1	Sarah Okafor	DevOps	50000
1	Sarah Okafor	Kubernetes	75000
2	James Adeyemi	DevOps	50000
2	James Adeyemi	DevOps	50000

- ✗ Repeated student and course data.
- ✗ Hard to update (update anomalies).
- ✗ Increases the risk of inconsistent data.

After: Normalized Design (Clean and Reliable)

students

student_id	full_name
1	Sarah Okafor
2	James Adeyemi

courses

course_id	course_title
101	DevOps
102	Kubernetes

enrollments

enroll_id	student_id	course_id
1	1	101
2	1	102
3	2	101

payments

payment_id	enroll_id	amount
1	1	50000
2	2	75000
3	3	50000

Relationships and Constraints



- ✓ **FOREIGN KEY** links related tables (e.g., enrollments.student_id → students.student_id).
- ✓ **UNIQUE** prevents duplicate values (e.g., a student email should be unique).
- ✓ These constraints keep your data accurate and consistent.

Practice



- ✓ Identify duplicated data in a given table.
- ✓ List the issues it can cause (e.g., update anomalies).
- ✓ Redesign it into normalized tables (students, courses, enrollments, payments).



Junior Engineer Insight

Normalize data for **consistency**, then evaluate performance using **evidence** (e.g., query time, EXPLAIN, and real usage patterns).

Secure MySQL Users and Permissions

Key Concepts

- MySQL has its own **ACCOUNTS** (users) that are different from **OPERATING-SYSTEM (OS)** users.
- Each MySQL user authenticates with a username and password.
- Users are given **PRIVILEGES** (permissions) to control what they can do (e.g., **SELECT**, **INSERT**, **UPDATE**).

OS Users vs MySQL Accounts

Aspect	Operating-System User	MySQL Account
Scope	Logs in to the server (e.g., ubuntu, ec2-user).	Logs in to MySQL (e.g., app_user).
Purpose	Manage the server (OS, files, services).	Access and manage databases and data.
Example	ubuntu (Linux user)	app_user@localhost (MySQL user)

Common MySQL Privileges

Privilege	What It Allows
SELECT	Read data from tables.
INSERT	Add new records.
UPDATE	Change existing records.
DELETE	Remove records.
CREATE	Create tables and other objects.
ALTER	Modify table structure.
DROP	Delete tables or other objects.

Create a Limited Application User

```
-- Step 1: Create the user
CREATE USER 'app_user'@'localhost'
IDENTIFIED BY 'StrongP@ssw0rd!';
-- Step 2: Grant only required privileges
GRANT SELECT, INSERT, UPDATE, DELETE
ON school_db.* TO 'app_user'@'localhost';
-- Step 3: Apply the changes
FLUSH PRIVILEGES;
```

Check User Permissions

```
SHOW GRANTS FOR 'app_user'@'localhost';
```

Example Output

```
Grants for app_user@localhost
GRANT SELECT, INSERT, UPDATE, DELETE
ON `school_db`.* TO 'app_user'@'localhost'
```

Why Not Use the Root User?

Using Root (Bad)

- Too much access (full control).
- Higher risk if the app is compromised.
- Can accidentally delete or change everything.

Use a Limited User (Good)

- Only the required permissions.
- Reduces the impact of a security breach.
- Safer and more controlled for applications.

Practice



- Log in with app_user and run a permitted query (e.g., **SELECT**).
- Try a restricted operation (e.g., **DROP TABLE**) and confirm it fails with a "Access denied" error.
- Verify the user's privileges using **SHOW GRANTS**.

Security Reminder



- Restrict network access (e.g., allow only trusted IPs).
- Use **STRONG, UNIQUE** passwords.
- Avoid hardcoded secrets in your code (use environment variables or a secrets manager).



Junior Engineer Insight

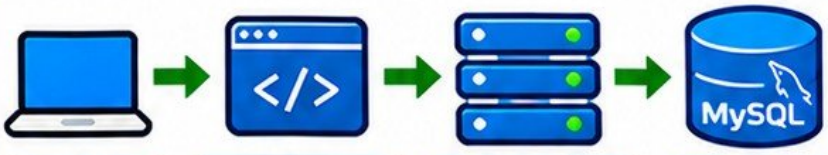
Use dedicated, least-privileged MySQL users for applications, and verify permissions regularly.

Connect an Application to MySQL

Key Concepts

- Applications (e.g., web APIs) connect to MySQL using a **DATABASE CONNECTION** (connection string) with the host, port, database, user, and password.
- Use **PARAMETERIZED QUERIES** to prevent **SQL INJECTION**.
- A **CONNECTION POOL** reuses database connections for better performance.

How the Connection Works



Client (Browser/ Mobile) → API (Backend App e.g., Python FastAPI) → Connection Pool (Reuses DB connections) → MySQL (Database Server)

Connection String Basics

Item	Description
Hostname	Address of the database server (e.g., 127.0.0.1, db, or mysql).
Port	Network port MySQL listens on (default: 3306).
Database	The specific database to use (e.g., school_db).
Username	A MySQL user with the right privileges (e.g., app_user).
Password	The user's password (keep it secret).

Example: Python Connection (mysql-connector)

```
import mysql.connector
conn = mysql.connector.connect(
    host="db",          # or "127.0.0.1"
    port=3306,
    database="school_db",
    user="app_user",
    password="StrongP@ssw0rd!",
    pool_name="school_pool",
    pool_size=5         # connection pool
)
```

Parameterized Query (Prevent SQL Injection)

Unsafe (Do Not Use)

```
sql = f"SELECT * FROM students WHERE email = '{email}'"
```

Safe (Use Parameters)

```
sql = "SELECT * FROM students WHERE email = %s"
cursor.execute(sql, (email,))
```

Why Use a Connection Pool?

- Reuses existing connections (instead of creating a new one each time).
- Reduces connection overhead and latency.
- Handles higher traffic and multiple users.
- Improves application performance and stability.

Practice



- Install the Python MySQL driver: pip install mysql-connector-python.
- Create a config with: host, port (3306), database, user, and password.
- Connect to MySQL and run a test query (e.g., SELECT 1;).
- Confirm the connection is working (with a successful result).

Common Failure



localhost inside a container refers to that container itself, **NOT** automatically to the MySQL server. Use the correct database hostname (e.g., "db" in Docker Compose).

Junior Engineer Insight



Configure a secure, pooled connection with the right host, port, and credentials, and always use **parameterized queries**.

Back Up and Restore Your Database

Key Concepts

- Protect your data and keep your application running after a failure.
- Use **mysqldump** to create a **LOGICAL BACKUP** (SQL file) of a database.
- Always **TEST** a backup by restoring it.
- Understand **RPO** and **RTO** for recovery planning.

Backup vs Export vs High Availability

Term	What It Means
Backup	A copy of your data that you can restore after loss or corruption.
Export	A way to move data out (e.g., SQL/CSV), usually for migration or sharing.
High Availability (HA)	A setup (e.g., replication or a cluster) that keeps the database available if one server fails.

Backup Flow (Big Picture)



MySQL (veriqta_academy) → mysqldump (Create backup) → veriqta_academy.sql (Backup file)

Create a Backup With mysqldump

```
# Create a logical backup of veriqta_academy
mysqldump -u app_user -p \
  veriqta_academy \
  > veriqta_academy_$(date +%F).sql
# Example output
Enter password: ********
(backup file created: veriqta_academy_2026-11-10.sql)
```

Restore Into a Separate Test Database

```
# 1. Create a test database
mysql -u app_user -p -e \
 "CREATE DATABASE veriqta_academy_test;"
# 2. Restore the backup into the test database
mysql -u app_user -p veriqta_academy_test \
  < veriqta_academy_2026-11-10.sql
# 3. Verify the restore
mysql -u app_user -p -e \
 "SHOW TABLES FROM veriqta_academy_test;"
```

RPO and RTO (Recovery Targets)

Term	Meaning	Example
RPO	Maximum amount of data you can afford to lose.	RPO = 15 minutes (you can lose up to 15 minutes of data).
RTO	Maximum time to restore and be operational.	RTO = 1 hour (the system must be back within 1 hour).

Practice



- ☑ Create a backup of veriqta_academy using **mysqldump**.
- ☑ Restore it to veriqta_academy_test.
- ☑ Verify the data (e.g., **SHOW TABLES** and row counts).

Production Awareness



- ☑ Run backups on a schedule (e.g., daily).
- ☑ Store backups securely (e.g., another server or cloud storage).
- ☑ Monitor and test restoration regularly.
- ☑ Keep a clear RPO and RTO for your system.



Key Takeaway

A backup is not proven usable until you successfully restore it.

Troubleshoot Common MySQL Failures

Key Concepts

- Identify and fix common MySQL errors quickly.
- Use service status, logs, network checks, and permissions to diagnose issues.
- Investigate active connections with SHOW PROCESSLIST;
- Understand why queries can be slow and how to fix them.

Common Errors, Causes, and Fixes

Error (Symptom)	Common Cause	Fix
Access denied for user ...	Wrong username/password or missing privileges.	Check credentials and GRANT the required privileges.
Unknown database ...	Database name is incorrect or not created.	Verify the name and CREATE DATABASE (if needed).
Table doesn't exist ...	Table name is wrong or not created.	Check the name and CREATE TABLE (if needed).
Can't connect to MySQL server ...	Service not running, wrong host/port, firewall, or network issue.	Check service status, port (3306), firewall, and network reachability.

Check the Basics First

- Service Status: Is MySQL running?
  ```
  systemctl status mysql
  ```
- Logs: Look for errors in the log file.
  ```
  sudo tail -n 50 /var/log/mysql/error.log
  ```
- Network: Can you reach the server (host and port)?
  ```
  ping <db-host>
  nc -vz <db-host> 3306
  ```
- Permissions: Does the user have the right privileges?
  ```
  SHOW GRANTS FOR 'app_user'@'%';
  ```

Investigate Active Connections

```
SHOW PROCESSLIST;
```

Id	User	Host	db	Command	Time	State
12	app_user	10.0.0.5	school_db	Query	0	NULL
18	app_user	10.0.0.5	school_db	Query	156	Sending data

Why Queries Can Be Slow

- Missing indexes (full table scans).
- Large tables or too much data.
- Poorly written queries (e.g., unnecessary joins).
- Many concurrent connections.
- Insufficient server resources (CPU, memory, disk).



Troubleshooting Flow (Step-by-Step)

Symptom (e.g., "Access denied") → Evidence (e.g., error message, logs, status) → Hypothesis (e.g., wrong credentials or missing rights) → Check (e.g., SHOW GRANTS, service status, config) → Fix (e.g., correct password or GRANT privileges) → Verify (e.g., run the query again successfully)

Practice



- Simulate a failed connection (e.g., wrong password).
- Use the troubleshooting flow to identify the cause.
- Apply the fix (e.g., correct the password or grant rights).
- Verify by connecting and running a simple query (e.g., SELECT 1;).

Production Awareness



- Monitor MySQL service and logs (e.g., with monitoring tools).
- Set alerts for failures and slow queries.
- Keep queries, indexes, and permissions well maintained.

Key Takeaway | Troubleshooting MySQL is a structured process: observe, test, fix, and always verify the result.

Run MySQL in Docker and Understand Production Operations

Key Concepts

- Run MySQL in a **DOCKER CONTAINER** for an isolated, reproducible database.
- Use a **PERSISTENT VOLUME** so data is not lost when the container is deleted.
- Configure MySQL with **ENVIRONMENT VARIABLES** (e.g., root password, database, user).
- Understand basic **PRODUCTION OPERATIONS**: health checks, monitoring, resource limits, and backups.

Architecture: App + MySQL + Persistent Storage



Application Container (e.g., FastAPI) → Docker Network (e.g., app-net) → MySQL Container (mysql:8.x) → Persistent Volume (MySQL Data)

Important MySQL Environment Variables

Variable	Purpose
MYSQL_ROOT_PASSWORD	Sets the root user password.
MYSQL_DATABASE	Creates the initial database (e.g., veriqta_academy).
MYSQL_USER	Creates an application user (e.g., app_user).
MYSQL_PASSWORD	Sets the password for the application user.

Run with Docker Compose (Example)

```yaml
version: "3.9"
services:
  db:
    image: mysql:8.0
    container_name: veriqta-mysql
    environment:
      MYSQL_ROOT_PASSWORD: StrongRoot@123
      MYSQL_DATABASE: veriqta_academy
      MYSQL_USER: app_user
      MYSQL_PASSWORD: App@123
    volumes:
      - mysql_data:/var/lib/mysql
    ports:
      - "3306:3306"
networks:
  app-net:
    driver: bridge
volumes:
  mysql_data:
```

Useful Docker Operations

Command	What It Does
docker ps	List running containers.
docker logs veriqta-mysql	View MySQL container logs.
docker exec -it veriqta-mysql \ mysql -u app_user -p	Open a MySQL shell inside the container.
docker stop veriqta-mysql	Stop the container (data remains).
docker rm veriqta-mysql	Remove the container (persistent data stays in volume).

Production Operations Checklist

- ✅ **Health Checks:** Ensure MySQL is ready and responding (e.g., **mysqladmin ping** or a Docker healthcheck).
- ✅ **Monitoring:** Track key metrics (CPU, memory, disk, connections).
- ✅ **Resource Limits:** Set CPU and memory limits (e.g., **--cpus=1 --memory=1g**) to protect the host.
- ✅ **Backups:** Run regular dumps (e.g., **mysqldump**) and store backups securely (e.g., another volume or object storage).

Why Data Survives Container Deletion



Container (Temporary) — Delete Container — Persistent Volume (Data Stays)

✅ The container can be removed and recreated, but the data in the volume (mysql_data) remains.

Practice



- ✅ Start the MySQL container using Docker Compose.
- ✅ Verify it is running with **docker ps**.
- ✅ Connect and list databases (e.g., **SHOW DATABASES;**).
- ✅ Delete the container and recreate it, then confirm that your data is still present.

Production Awareness

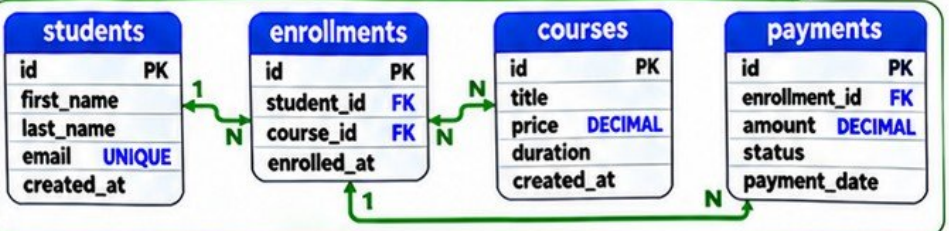
A single MySQL container is useful for learning and development, but it is **NOT** automatically a **highly available** production database.

Final Project: Build and Defend Your First MySQL Database

Project Scenario

VERIQTA Academy needs a database to manage students, courses, enrollments, and payments.

Database Design (Entities and Relationships)



Tasks You Must Complete

- Create the veriqta_academy database and the four related tables.
- Apply PRIMARY KEYS, FOREIGN KEYS, and appropriate DATA TYPES.
- Insert realistic test records.
- Write queries using WHERE, JOIN, GROUP BY, and ORDER BY.
- Update a record safely and verify it.
- Demonstrate a TRANSACTION ROLLBACK.
- Create a LEAST-PRIVILEGE application user.
- Back up the database and TEST RESTORATION.
- Investigate one simulated connection or query failure.
- Explain how the database would connect to a real application.

Key Implementation Examples

1. Create Database & Tables (Sample)

```
CREATE DATABASE veriqta_academy;
USE veriqta_academy;
CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  created_at DATETIME DEFAULT NOW()
);
```

2. Sample Query (JOIN + WHERE)

```
SELECT s.first_name, c.title,
       p.amount, p.status
FROM enrollments e
JOIN students s ON e.student_id = s.id
JOIN courses c ON e.course_id = c.id
JOIN payments p ON p.enrollment_id = e.id
WHERE p.status = 'paid'
ORDER BY p.payment_date DESC;
```

3. Transaction & Rollback

```
START TRANSACTION;
INSERT INTO payments
  (enrollment_id, amount, status)
VALUES (1, 250.00, 'pending');
ROLLBACK;  -- Change not saved
```

4. Backup & Restore

```
# Backup
mysqldump -u app_user -p \
  veriqta_academy > academy_backup.sql

# Restore (to a test DB)
mysql -u app_user -p \
  veriqta_academy_test < academy_backup.sql
```

Test Data (Example)

Table	Example Records (Sample)
students	(1, 'Ada', 'Okoro', 'ada@veriqta.com')
courses	(1, 'DevOps Basics', 250.00, '4 Weeks')
enrollments	(1, 1, 1, '2026-11-01')
payments	(1, 1, 250.00, 'paid', '2026-11-01')

Verifications (What to Show)

- Tables created with correct keys and data types.
- Data relationships (e.g., a student has enrollments and payments).
- Query results for filtering, joining, grouping, and sorting.
- Update verified (e.g., change payment status to 'paid').
- Rollback verified (e.g., record does not exist after rollback).
- Application user with only needed privileges (SHOW GRANTS).
- Backup file created and successfully restored to a test database.

Real Application Connection (Example)

```
conn = mysql.connector.connect(
  host="db",      # e.g., Docker service name
  port=3306,
  database="veriqta_academy",
  user="app_user",
  password="StrongP@ssw0rd"
)
```

Troubleshooting Example

- Failure: Can't connect to MySQL server.
- Check: Service status, host, port (3306), and user.
- Fix: Start MySQL, use the correct hostname, and verify credentials.
- Verify: Connect and run SELECT 1;

Final Verification

You should be able to demonstrate correct data relationships, query results, access controls, and a successful backup restoration.

VERIQTA

Senior DevOps Resume Playbook

Build a Production-Ready Resume That Gets Interviews, Not Just Views



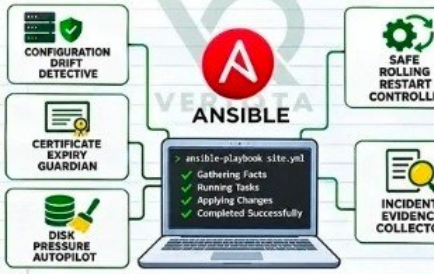
Turn Your Experience Into Senior-Level Evidence

- Real Achievement Examples
- Quantifiable Impact & Metrics
- ATS-Ready Resume Structure
- Role-Specific Resume Versions (DevOps, SRE, Platform)
- Portfolio & GitHub Strategy
- Interview-Ready Proof of Work

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

5 Ansible Playbooks That Automate Your Busywork



- Real-World Automation for Operations, Reliability, and Speed
- 5 Production-Ready Playbooks with Code and Use Cases
- Save Time, Reduce Errors, and Keep Systems Healthy

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

10 Kubernetes Projects That Will Get You Hired



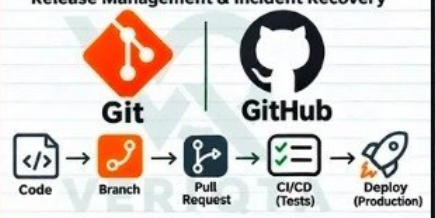
- Real-World Projects
- Production-Ready Designs
- Hands-On Configurations
- Senior Engineering Skills

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Git & GitHub: Production Engineering Playbook

Repository Architecture, CI/CD, Security, Release Management & Incident Recovery



- Core Git Workflows
- Collaborative Development
- Branching and Merging Strategies
- Code Review and Pull Requests
- CI/CD with GitHub Actions
- Release, Tagging and Rollback
- Security, Access and Governance
- Real-World Projects and Fixes



@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Build a Disaster Recovery Lab: Back Up, Destroy, and Restore a Real Application



RTO / RPO Planning | Real Application Lab | Disaster Recovery Exercises | Verification & Recovery Runbook

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Cloud Troubleshooting Commands Handbook

20 Production Incident Investigations

Diagnose • Investigate • Fix • Verify

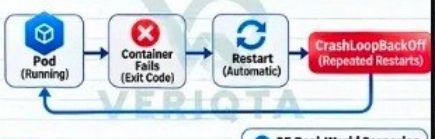


Real-World Cloud Scenarios for DevOps, SRE, and Platform Engineers

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Kubernetes CrashLoopBackOff: Diagnose 25 Real Container Restart Failures




```
$ kubectl get pods
NAME                   READY   STATUS             RESTARTS
web-7d9f6c8f7d-abc12   0/1     CrashLoopBackOff   18
```

- 25 Real-World Scenarios
- Step-by-Step Diagnosis
- Root Cause Analysis
- Proven Fixes
- Best Practices

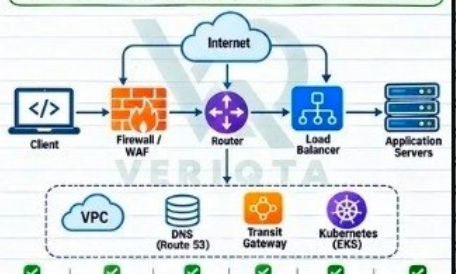
LOGS & EVENTS | PROBES (LIVENESS / READINESS) | RESOURCES (CPU / MEMORY) | CONFIG & SECRETS | DEPENDENCIES (DNS / DB / API)

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

50 Networking Problems for DevOps Engineers

Real Production Incidents, Root Causes and Fixes



Connectivity Failures | DNS & Name Resolution Issues | Routing and Traffic Problems | Load Balancing Issues | Kubernetes Networking Problems | Real Troubleshooting and Fixes

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

How to Debug a Broken Server Like a Detective

Real-World Troubleshooting for Linux, Nginx, Cloud and Production Systems



Linux Troubleshooting | Nginx Diagnostics | Database Issues | Cloud & Networking | Container & Deployment | Monitoring & Recovery

30 Real-World Cases, Diagnoses and Fixes

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Senior DevOps Resume Playbook

Build a Production-Ready Resume That Gets Interviews, Not Just Views



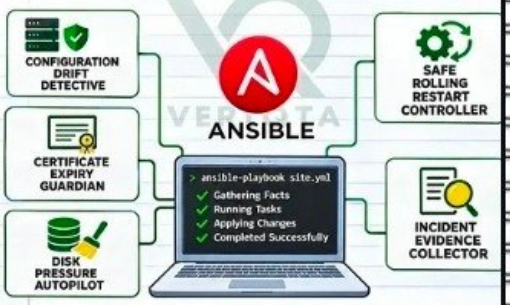
Turn Your Experience Into Senior-Level Evidence

- Real Achievement Examples
- Quantifiable Impact & Metrics
- ATS-Ready Resume Structure
- Role-Specific Resume Versions (DevOps, SRE, Platform)
- Portfolio & GitHub Strategy
- Interview-Ready Proof of Work

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

5 Ansible Playbooks That Automate Your Busywork



- Real-World Automation for Operations, Reliability, and Speed
- 5 Production-Ready Playbooks with Code and Use Cases
- Save Time, Reduce Errors, and Keep Systems Healthy

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

10 Kubernetes Projects That Will Get You Hired



- Real-World Projects
- Hands-On Configurations
- Production-Ready Designs
- Senior Engineering Skills

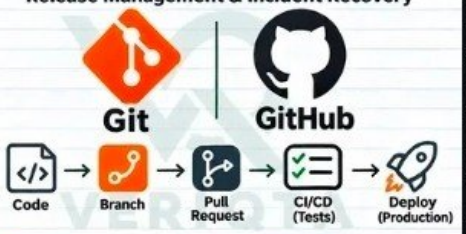
@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Git & GitHub:

Production Engineering Playbook

Repository Architecture, CI/CD, Security, Release Management & Incident Recovery



- Core Git Workflows
- Collaborative Development
- Branching and Merging Strategies
- Code Review and Pull Requests
- CI/CD with GitHub Actions
- Release, Tagging and Rollback
- Security, Access and Governance
- Real-World Projects and Fixes

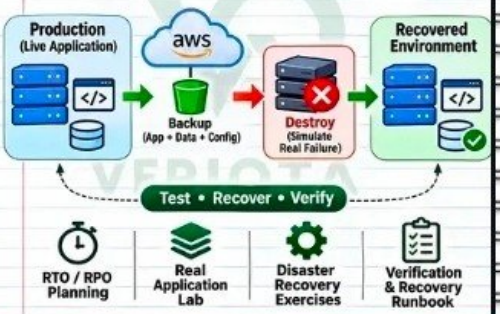


@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Build a Disaster Recovery Lab:

Back Up, Destroy, and Restore a Real Application



RTO / RPO Planning | Real Application Lab | Disaster Recovery Exercises | Verification & Recovery Runbook

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Cloud Troubleshooting Commands Handbook

20 Production Incident Investigations

Diagnose • Investigate • Fix • Verify



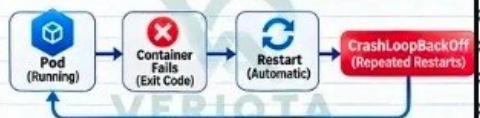
Real-World Cloud Scenarios for DevOps, SRE, and Platform Engineers

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

Kubernetes CrashLoopBackOff:

Diagnose 25 Real Container Restart Failures





- 25 Real-World Scenarios
- Step-by-Step Diagnosis
- Root Cause Analysis
- Proven Fixes
- Best Practices

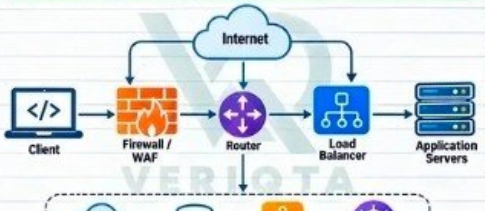
LOGS & EVENTS | PROBES (LIVENESS / READINESS) | RESOURCES (CPU / MEMORY) | CONFIG & SECRETS | DEPENDENCIES (DNS / DB / API)

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

50 Networking Problems for DevOps Engineers

Real Production Incidents, Root Causes and Fixes



Connectivity Failures | DNS & Name Resolution Issues | Routing and Traffic Problems | Load Balancing Issues | Kubernetes Networking Problems | Real Troubleshooting and Fixes

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

How to Debug a Broken Server Like a Detective

Real-World Troubleshooting for Linux, Nginx, Cloud and Production Systems



Linux Troubleshooting | Nginx Diagnostics | Database Issues | Cloud & Networking | Container & Deployment | Monitoring & Recovery

30 Real-World Cases, Diagnoses and Fixes

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta